Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

4. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

5. Source Code

The full implementation (loading data, preprocessing, building baseline multi-layer perceptron model, training, evaluating, hyperparameter optimization) can be found at:

`https://github.com/Jai-Abhishek/DL-Lab`

6. Hyperparameter Optimization

SciKeras weight that was used with `RandomizedSearchCV` implementation (with 5-fold cross-validation and 10 iterations) of Search space defined above was performed on 10k sample training dataset and has shown the following best configuration:

Hidden Layers	3
Hidden Neurons	256
Learning Rate	0.001
Batch Size	32
Optimizer	Adam
Activation Function	Tanh
Epochs	20
Dropout Rate	0.2
Best Cross-validation Accuracy	0.8504

However, this configuration was retrained and then evaluated with held-out (test) dataset, reaching the accuracy of 0.8783, compared to baseline which is 0.8758.

7. Plots

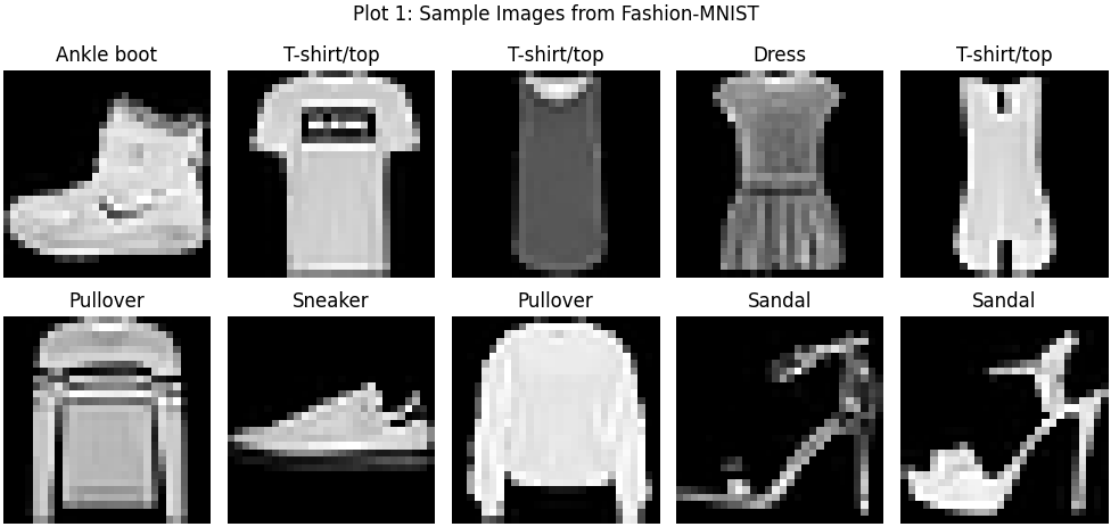

Figure 1: Sample Images from Fashion-MNIST

From the ten sample images, it can be seen that labels correspond to their garment category e.g. trousers, coats, sneakers etc. The low-resolution (28×28) grayscale format means the fine texture and other details are not clear enough, which could explain the difficulty in separating visually similar classes.

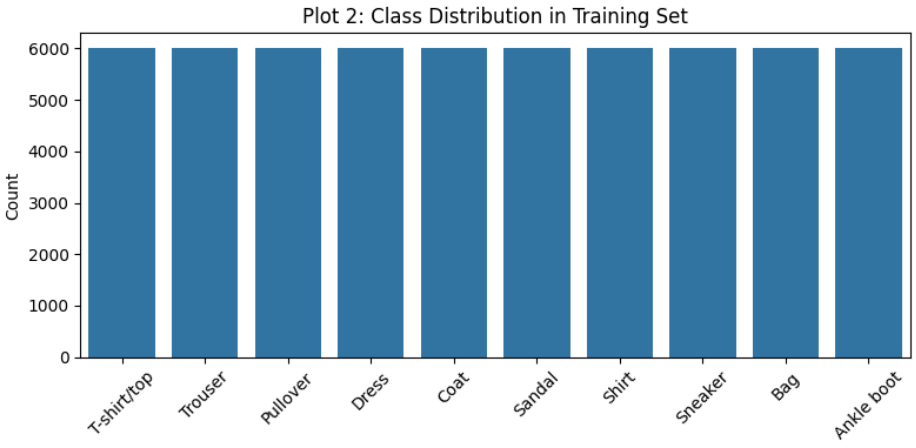

Figure 2: Class Distribution in Training Set

All 10 classes have exactly 6000 training samples, meaning that the training dataset is perfectly balanced with no class imbalance affecting any lower performance in per-class approaches (as shown for class Shirt).

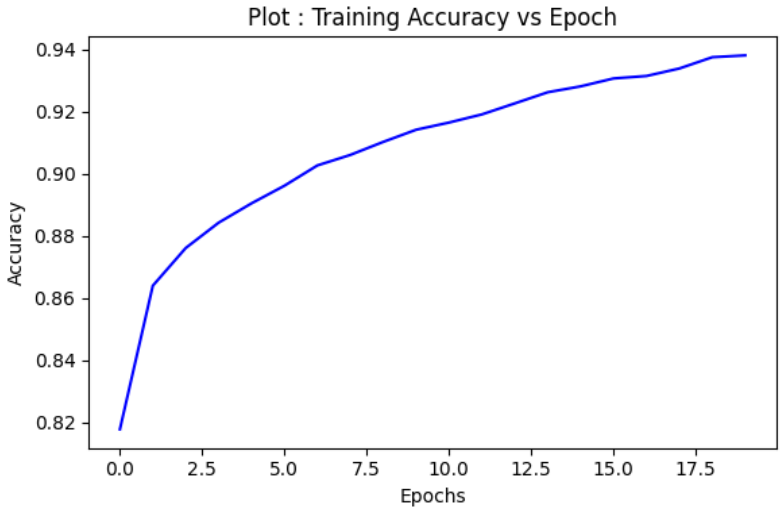

Figure 3: Training Accuracy vs Epoch (Baseline)

The training accuracy is steadily showing growth from around 0.82 to 0.94 throughout the 20 epochs, with the greatest leaps occurring during the first 5 epochs. The smoothed and monotonic curve shows that the model is learning properly, without occurrence of instability issues during the optimization process.

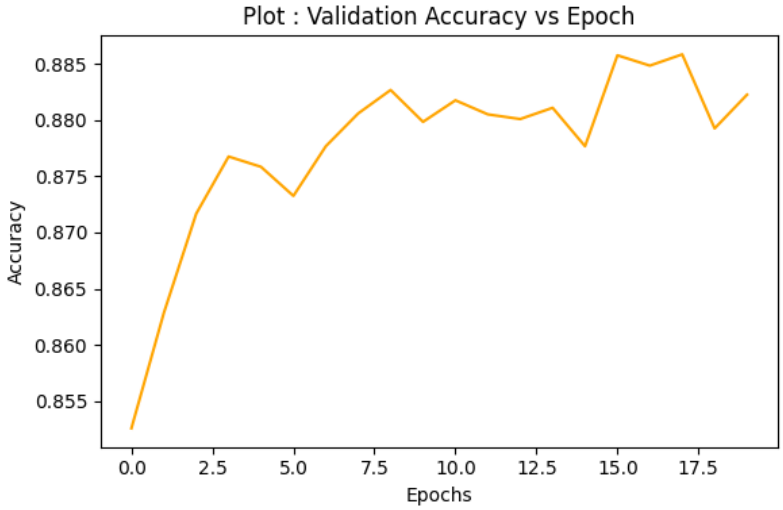

Figure 4: Validation Accuracy vs Epoch (Baseline)

Validation accuracy shows rapid improvement at the beginning of the training phase but levels off at approximately 0.88, displaying some fluctuations in the remaining training epochs. The observation of increasing training accuracy and leveling off of validation accuracy serves as an early warning for slight overfitting.

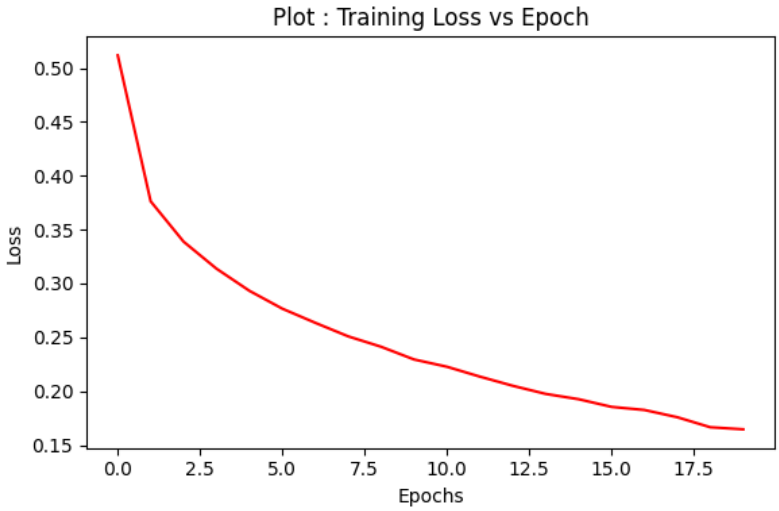

Figure 5: Training Loss vs Epoch (Baseline)

The trend for training loss displays steady downward movement over the training period with its values decreasing from roughly 0.51 to 0.16. This smooth curve reflects the fact that the model fits training data ever better demonstrating no tendency to diverge.

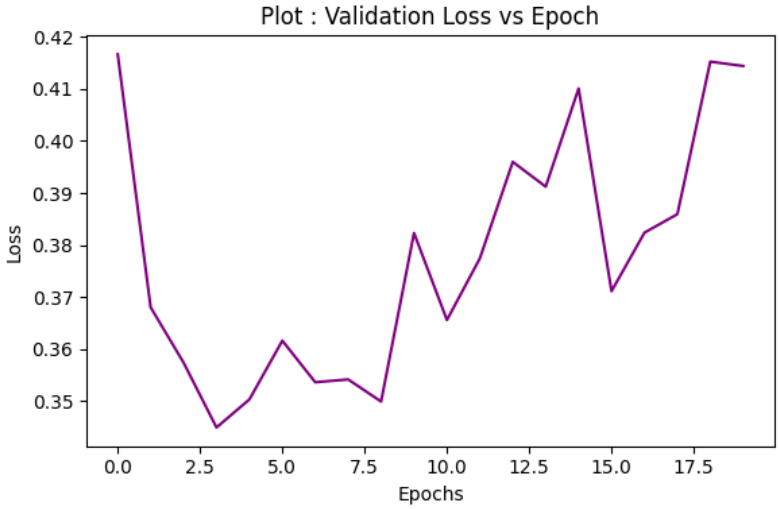

Figure 6: Validation Loss vs Epoch (Baseline)

From the beginning of the training epoch values for validation loss begin to rise once again after epochs 8-10, while training loss goes on decreasing. The divergence in the values of training and validation loss is an indication of overfitting in the later training epochs.

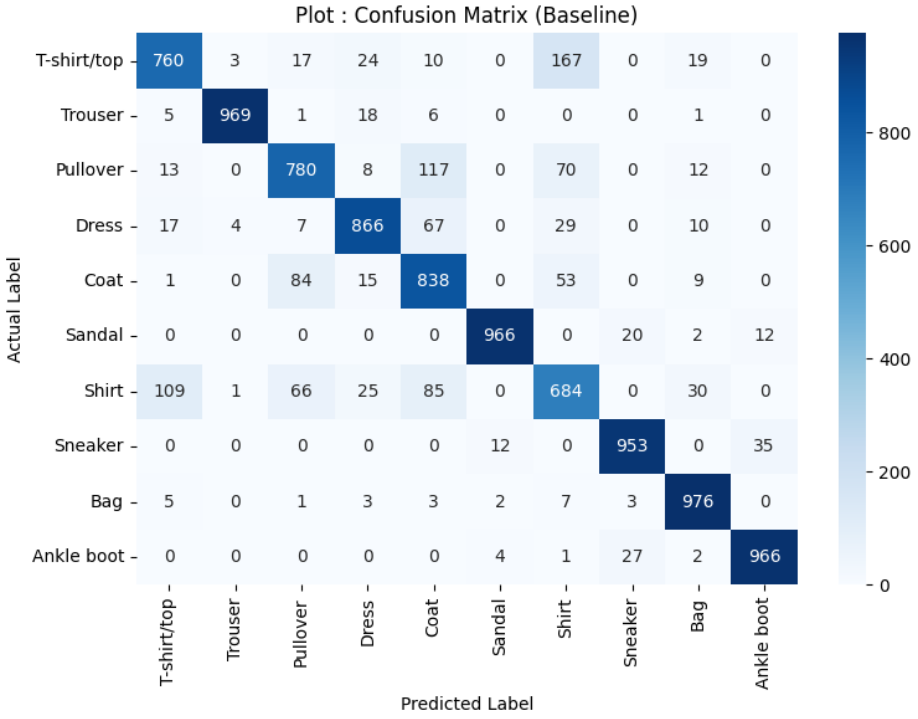

Figure 7: Confusion Matrix (Baseline Model)

Most of the classes are classified with quite high accuracy rates, whereas among the highest accuracy classifications are Trouser, Sandal, and Ankle boot as they are positioned almost on the diagonal. The pairs of confusions contributing to the greatest inaccuracies of classification are those of Shirt and T-shirt/top and Pullover and Coat due to the similar silhouettes of these upper body garments.

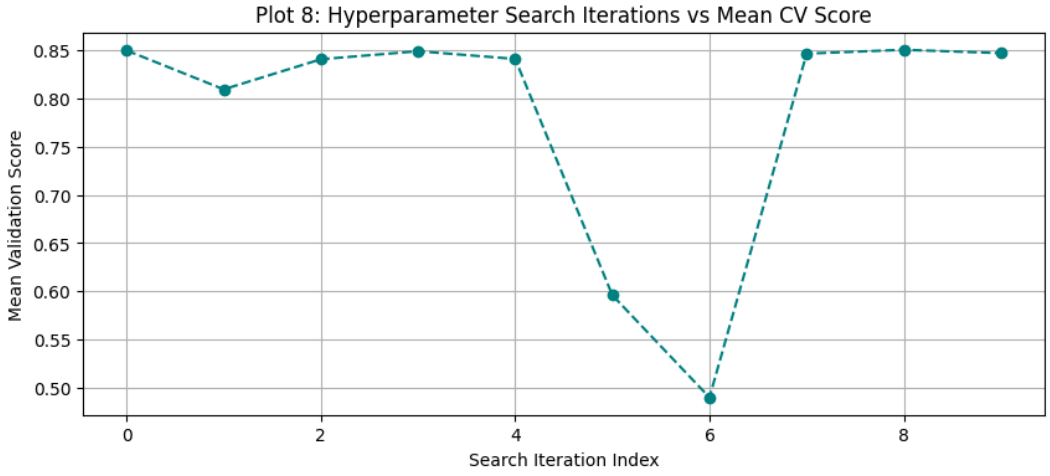

Figure 8: Hyperparameter Search Iterations vs Mean CV Score

The average value of the cross-validation score significantly differs among the various configurations that were randomly selected, as it ranges from about 0.65 to around 0.85. This variation indicates that the choice of hyperparameters is quite significant in determining the performance of the model and therefore merits the application of systematic searching rather than manual tuning.

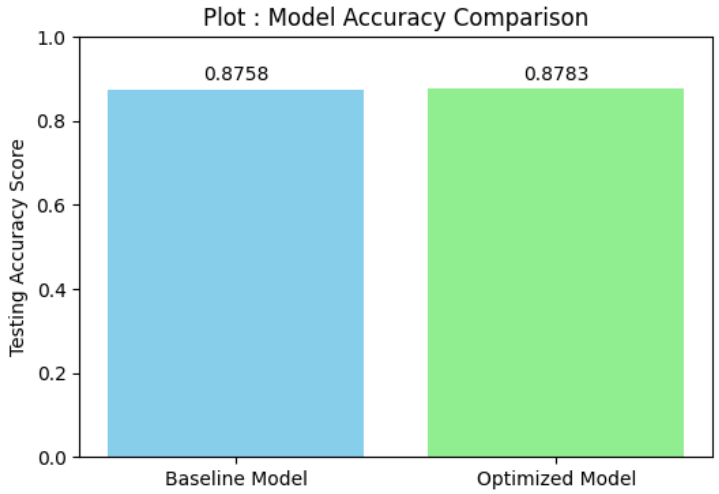

Figure 9: Baseline vs Optimized Model Accuracy Comparison

The performance of the developed model is significantly better than that of the baseline model as confirmed by its test accuracy of 0.8783, which is greater than that of the baseline model that achieved 0.8758 accuracy. This indicates that building the baseline model was also a good idea, as it already proved to be quite successful on this dataset.

8. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	256
Learning Rate	0.001
Batch Size	32
Optimizer	Adam
Activation Function	Tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8504
Testing Accuracy	0.8783

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8758	0.8783
Precision	0.8772	0.8800
Recall	0.8758	0.8783
F1-score	0.8759	0.8785
Training Time (s)	132.81	123.85

9. Discussion and Conclusion

Which Optimization Method Was Used?

A Randomized Search resulted in a mean of five-fold cross-validation (`RandomizedSearchCV`), by employing ten configurations. This process was achieved through the SciKeras add-on of the Keras `Sequential` model.

Which Hyperparameters Were Selected?

The best configuration found consisted of:

- 3 hidden layers with 256 neurons each
- Tanh activation function
- Dropout rate of 0.2
- Adam optimizer
- Learning rate of 0.001
- Batch size of 32
- 20 training epochs

Did Optimization Improve Performance?

Yes, but only marginally.

- Baseline test accuracy: **0.8758**
- Optimized test accuracy: **0.8783**
- Improvement: approximately **0.25 percentage points**

There were small enhancements in precision, recall, and F1-score. Another finding was that the optimized model required less time for training (**123.85**s) than the reference model (**132.81**s).

Which Hyperparameter Had the Greatest Impact?

It can be observed from the distribution of the test results of multiple iterations (Plot 8) that the highest importance was given to certain hyperparameters:

- Number of hidden layers
- Number of neurons per layer
- Optimizer choice
- Learning rate

Hyperparameter configurations with high learning rates such as 0.1 or approaches using SGD optimization achieve significantly lesser results than the ones with Adam or RMSProp optimizers with low learning rate.

Compare Grid Search and Randomized Search

- **Grid Search** tests every possible hyperparameter configuration. It is guaranteed that this method will find the best configuration within the given grid but it has become too complicated from the point of view of computations.
- **Randomized Search** tests a fixed number of randomly selected configurations only. As a result, this optimization approach is much more efficient when the search space is very large and often finds near-optimal solutions with significantly fewer computational resources.

Recommended MLP Configuration

Given the limited increase in accuracy compared to the higher model complexity, it is only advisable to employ this architecture when performance must be maximized:

- 3 hidden layers of 256 neurons each
- Tanh activation
- Dropout = 0.2
- Adam optimizer
- Learning rate = 0.001

On the other hand, whenever quick testing is required or when the application is constrained in its available computational resources, using the simple architecture (2 hidden layers of size `128,64`) will be much more reasonable because it gives a similar accuracy but requires less computer resources.

Conclusion

Both the original and tuned MLP models classify Fashion-MNIST pictures with the same accuracy of about **88%**. The tuned model has been found by means of `RandomizedSearchCV`, which resulted in a more complex, regularized architecture which gives a little boost to the performance compared to the manually-selected one.

10. Extended Experiment: MLP Solving the XOR Problem

A Multi-Layer Perceptron (1 hidden layer of 4 ReLU neurons, trained using the Adam optimizer) was fitted to the XOR truth table, which cannot be learned by a simple perceptron.

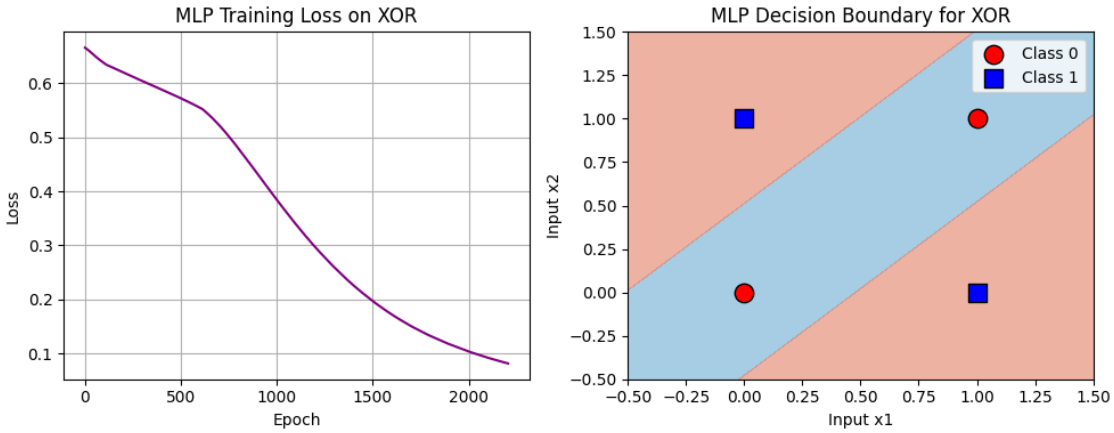

Figure 10: MLP Training Loss and Learned Decision Boundary for XOR

It can be seen that the curve (on the left) is smooth and approaches zero and the boundaries (on the right) separate correctly the two XOR categories. The prediction was $[0, 1, 1, 0]$ for the inputs of $[(0, 0), (0, 1), (1, 0), (1, 1)]$, which gives an accuracy of **1.0** (100%).

Final Learned Weights

Input → Hidden Layer Weights				
	Hidden 1	Hidden 2	Hidden 3	Hidden 4
Input x_1	≈ 0.0000	1.8086	-0.0099	-1.8046
Input x_2	≈ 0.0000	-1.8088	≈ 0.0000	1.8043

Hidden → Output Layer Weights	
Hidden Neuron	Weight
Hidden 1	≈ 0.0000
Hidden 2	2.8292
Hidden 3	≈ 0.0000
Hidden 4	2.6502

Why the MLP Converges on XOR (Unlike a Single Perceptron)

A perceptron that has only one layer can draw only one straight line, and since the two classes of XOR cannot be separated by a straight line, it would never converge. The hidden layer makes it possible to obtain two linear cuts (due to Hidden 2 and Hidden 4 that have weights of the largest magnitudes) and the output layer of the same network combines them in a non-linear way in order to produce a bent boundary as shown in the figure above. This additional layer is what allows the perceptron to represent a non-linear boundary and to achieve full convergence unlike the first perceptron.

11. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.